IDP Workflow Deep Dive

Visual diagrams and detailed request-by-request explanations for the actual Flask project

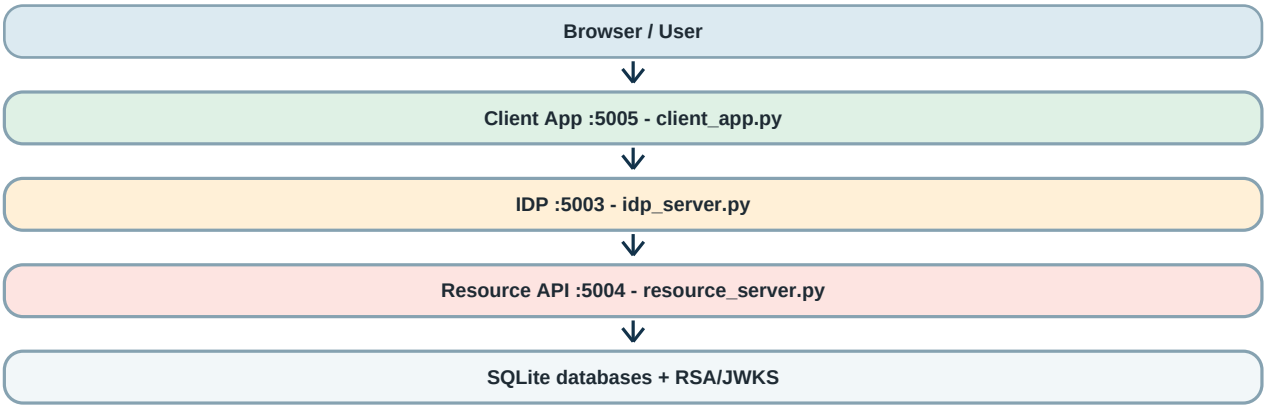
Repository: C:/Users/itzme/Desktop/All Stacks/03-IDP
Services: IDP :5003 | Resource API :5004 | Client App :5005

How to use this guide

Start with the architecture page, then follow the OAuth login workflow. The later pages zoom into registration, token creation, token validation, protected resources, refresh, logout, failures, and production hardening. Every workflow names the actual file/function responsible for the step.

1. System Architecture

The trust boundaries explain why the client, IDP, and resource server are separate.



What happens

The browser interacts with the client web app. The client redirects authentication to the IDP. The IDP reads identity data, signs tokens, and publishes its public key. The resource server receives Bearer access tokens and independently validates them before returning protected documents.

Why this split matters

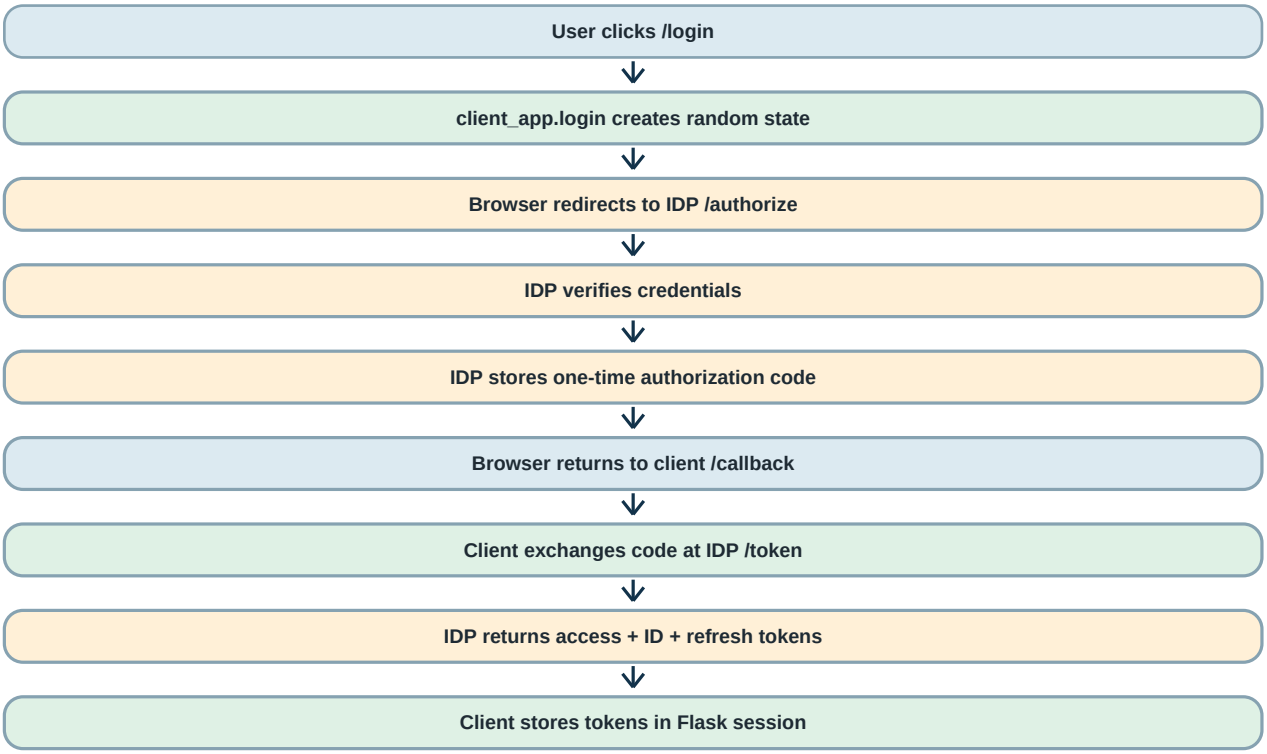
The resource server never receives the IDP private key. It can verify a signature with the public JWKS, but it cannot mint a valid token. This is the central asymmetric-trust idea in the project.

Source locations

Responsibility	Actual implementation
Identity and tokens	idp_server.py: init_db, /authorize, /token, create_access_token, create_id_token
Browser client	client_app.py: /login, /callback, /profile, /data, /logout
API protection	resource_server.py: get_signing_key, require_token, /api/profile, /api/data
Identity storage	idp_database.db: users, clients, auth_codes, refresh_tokens
Business data	resource_database.db: user_documents

2. OAuth Login: The Main Workflow

This is the path used when the user clicks Sign In in the client app.



Step-by-step

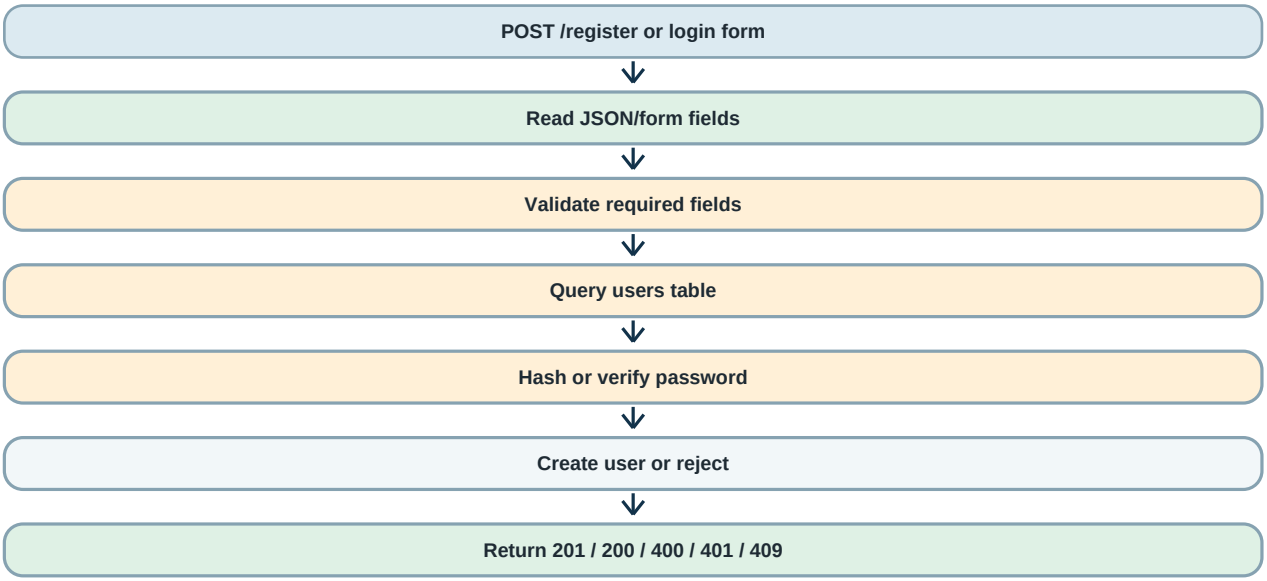
Step	What / why / where
1	client_app.py login() creates a cryptographically random oauth_state. It binds the response to the browser session and helps prevent login CSRF.
2	The client redirects to IDP /authorize with response_type=code, client_id, redirect_uri, scope, and state. The password is not sent to the client.
3	idp_server.py authorize() GET validates client_id and renders templates/login_form.html. POST checks the submitted username/password with check_password_hash().
4	After authentication, authorize() creates a random code and inserts it into auth_codes with client, redirect URI, scope, expiry, and used=0.
5	The IDP redirects to client /callback?code=...&state=.... The code is short-lived and intended for one exchange.
6	client_app.py callback() compares received state with session oauth_state, then makes a server-to-server POST /token request.
7	idp_server.py token() checks code existence, used flag, expiry, client ID, and user. It marks the code used before issuing tokens.
8	The client stores access_token, refresh_token, id_token, and decoded display data in the Flask session, then sends the user to /profile.

Failure branches

State mismatch redirects to a security error. Missing code or failed token exchange returns the user to the home page. Invalid credentials render the IDP login form with a 401 response. Reusing the authorization code returns invalid_grant.

3. Registration and Direct Login

The project supports both a direct JSON login API and registration through the IDP.



Registration: /register

Stage	Implementation detail
Input	username, email, password, optional full_name; JSON or form data.
Validation	Missing values -> 400 invalid_request. Password shorter than six characters -> 400 invalid_password.
Duplicate check	Parameterized query checks username OR email. Existing identity -> 409 user_exists.
Credential storage	generate_password_hash(password) creates the stored password_hash; the original password is not inserted.
Success	INSERT users and return 201 with id, username, email, and full_name; no password is returned.

Direct login: /login

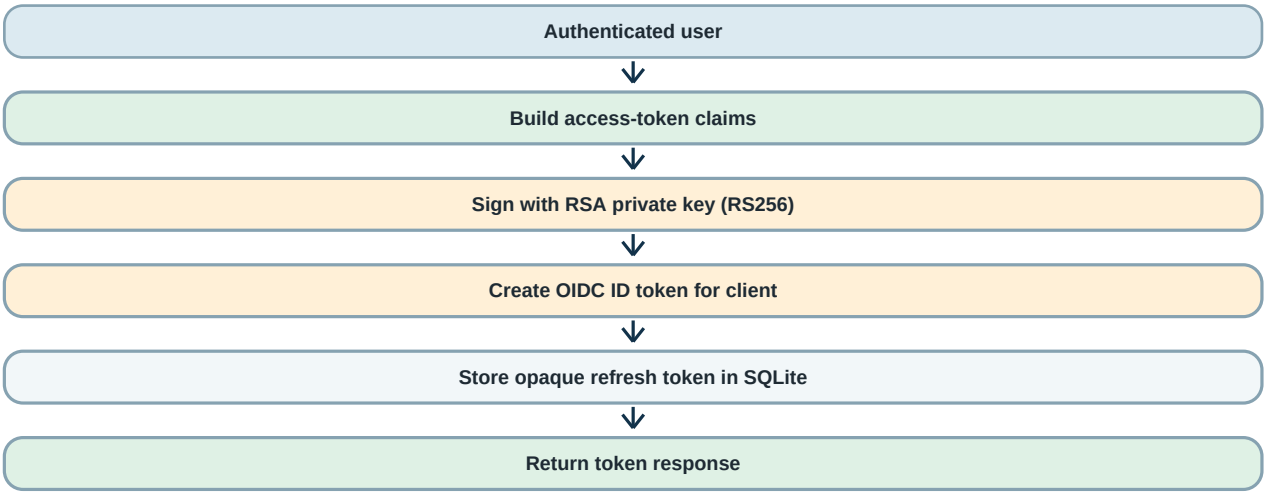
POST /login accepts username and password. idp_server.py login() loads the user by username, calls check_password_hash(), then creates an RS256 access token and an opaque refresh token. Success returns 200 with token_type=Bearer and expires_in=900; missing fields return 400; incorrect credentials return 401.

Important distinction

The browser's normal path is Authorization Code, not direct login. Direct /login is useful for testing and API consumers, but a production browser application should generally avoid collecting the user's password outside the IDP.

4. Token Creation and Token Contents

The IDP creates different credentials for different consumers.



Access token

create_access_token() sets iss=http://localhost:5003, sub=username, aud=resource_server, iat, exp=15 minutes later, user_id, email, name, and a space-separated scope string. The header includes alg=RS256, typ=JWT, and kid=idp-key-1.

ID token

create_id_token() uses the client ID as audience and carries identity claims such as sub, email, and name. It is meant for the client, not for authorizing resource API calls.

Refresh token

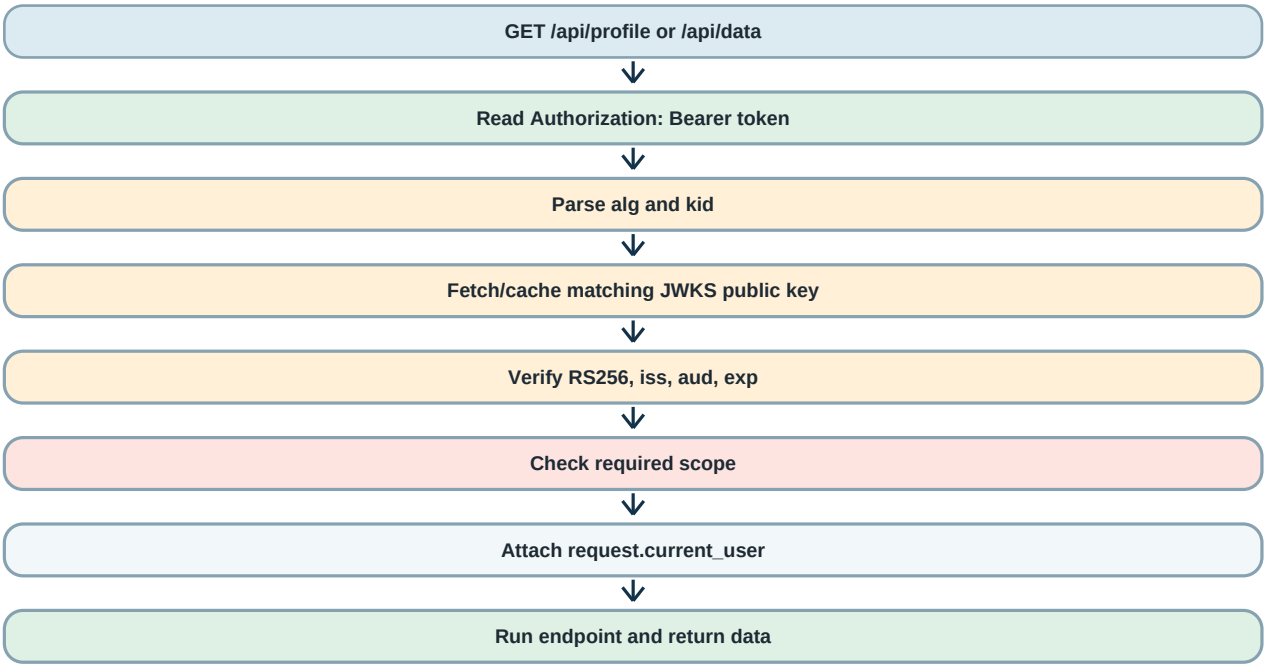
create_refresh_token() generates an opaque random value, stores it in refresh_tokens with username, client_id, scope, expiry seven days later, and revoked=0. It is looked up server-side and is not a JWT.

Security meaning

JWT payloads can be decoded by anyone who receives the token. The RS256 signature prevents undetected modification. Confidential data should not be placed in a JWT merely because it is signed.

5. Protected API Request and JWT Validation

The resource server performs authentication and authorization before the handler runs.



Validation sequence in require_token()

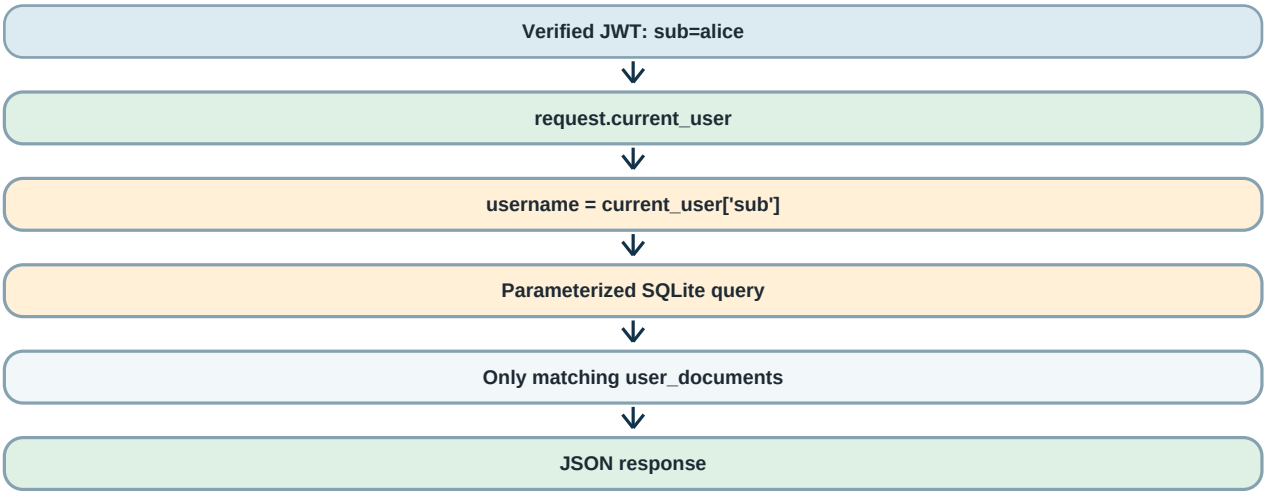
Check	Reason
Header format	Reject missing or malformed Bearer header with 401.
Algorithm	Only RS256 is accepted; this avoids accepting an unexpected algorithm.
Key ID	kid selects a public key from IDP /jwks. Keys are cached for five minutes.
Signature	PyJWT verifies the token was signed by the matching IDP private key.
Issuer and audience	iss must match the configured IDP and aud must be resource_server.
Expiration	exp is enforced; expired tokens return token_expired with 401.
Scope	profile/read/write is checked after token validity. Missing scope returns 403.

Endpoint behavior

GET /api/profile requires profile and returns identity claims from the verified token. GET /api/data requires read and queries documents where username equals the token subject. POST /api/data requires write, validates title/content, and inserts a document for the subject.

6. Protected Data Ownership

Authentication identifies the subject; the handler uses that subject to select private data.



Why the subject matters

The resource server does not trust a username supplied in the browser form for reads. It takes the subject from the verified access token. This prevents a caller from simply changing a request parameter to read another user's documents.

Read workflow

resource_server.py get_data() extracts sub, runs SELECT ... FROM user_documents WHERE username = ?, orders by id descending, and returns documents plus count. The database stores the ownership value on each row.

Write workflow

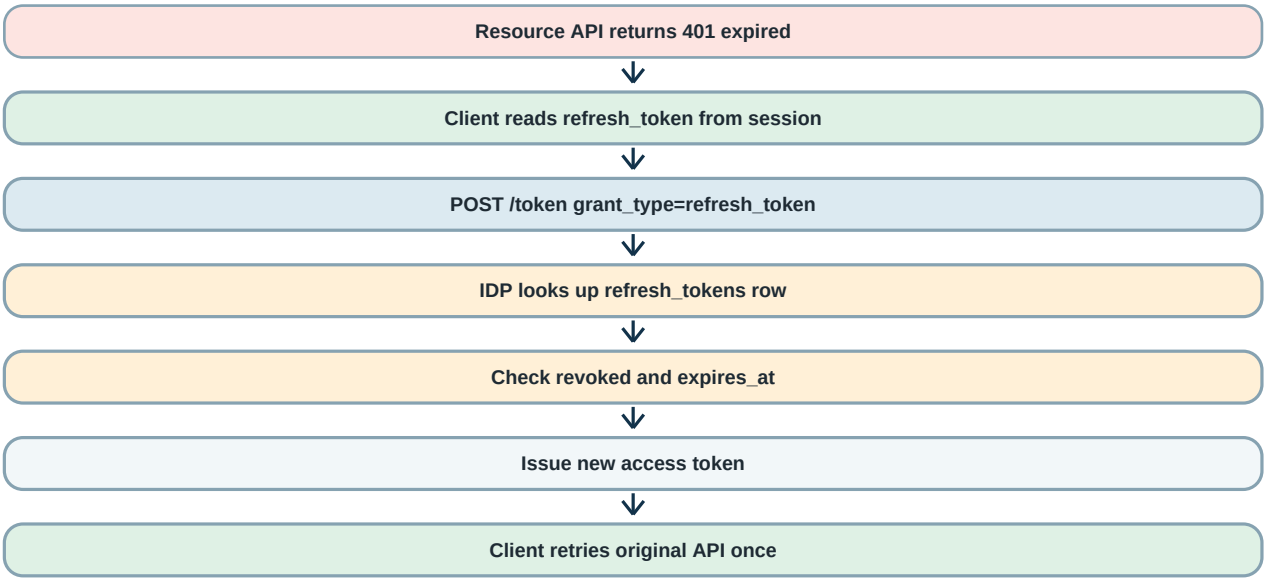
create_data() first passes token and write scope checks, then reads title/content/category from the request, rejects empty title/content, and inserts username from the token subject. It returns 201 with the created document.

Limitation

The database uses a username string as the ownership link rather than a foreign-key user ID. Production systems generally use stable immutable subject IDs, relational constraints, and stronger tenant isolation.

7. Refresh Token Workflow

Refresh keeps a session alive after the short-lived access token expires.



Automatic path

client_app.py make_authenticated_api_request() sends the access token. If the resource server returns 401, refresh_access_token() posts the refresh token to IDP /token and retries the original GET or POST once with the new access token.

IDP checks

idp_server.py token() or /token/refresh verifies that the opaque token exists, is not revoked, is not expired, and still maps to an existing user. It then creates a new access token using the stored scope.

What is not happening

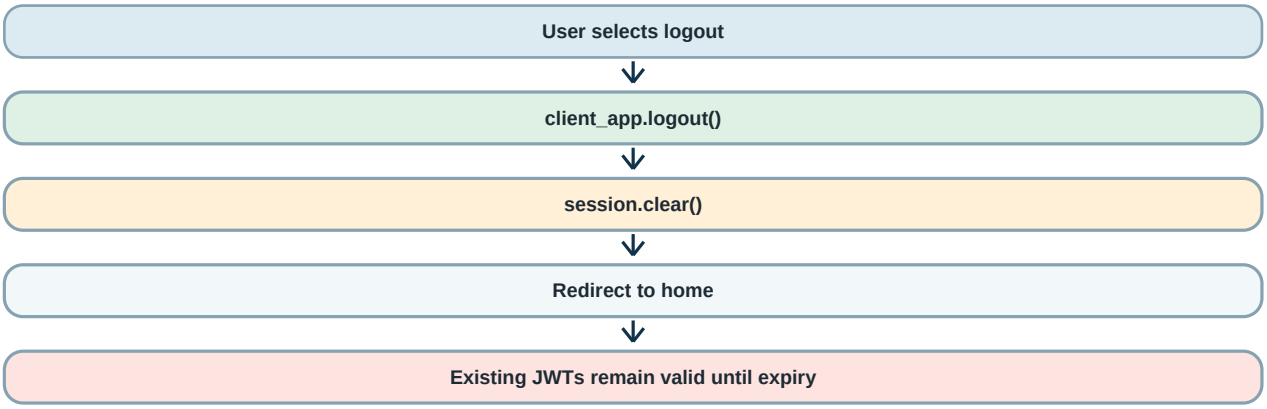
The current implementation does not rotate the refresh token on every use and does not revoke all refresh tokens during client logout. Those are production hardening items.

Failure

An invalid, expired, revoked, or userless refresh token returns invalid_grant. The client cannot refresh and eventually redirects the user to sign in again.

8. Logout and Error Workflows

Logout and error status codes describe different levels of trust failure.



Current logout behavior

The client clears its Flask session cookie state. The IDP is not called, refresh tokens are not revoked, and the resource server has no token blacklist. This is a local client logout, not global token revocation.

401 vs 403

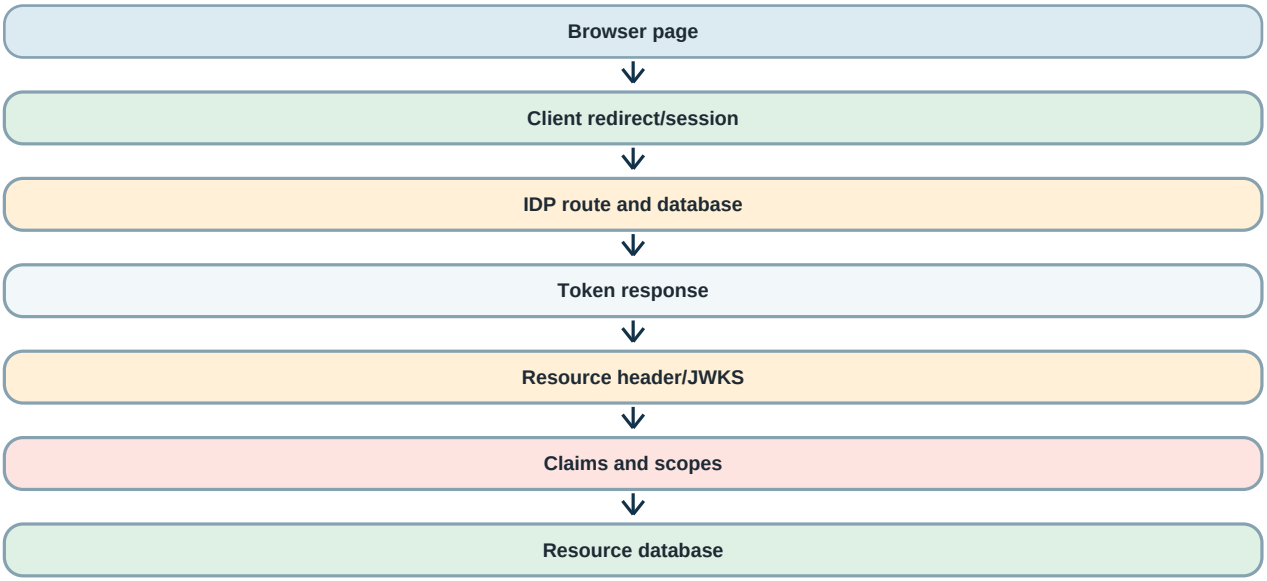
Status	Meaning in this project	Typical fix
400	Malformed request, missing field, unsupported grant, or expired authorization code in some flows.	Correct request parameters.
401	No credential, invalid password, malformed/expired JWT, wrong issuer/audience, or missing Bearer header.	Authenticate again, repair token/header, or refresh.
403	Valid token, but missing required scope such as read or write.	Request appropriate scope or change policy.
409	Username or email already exists.	Choose a different identity or use account recovery.

Production logout

A production design may revoke refresh tokens, terminate the IDP browser session, clear secure cookies, and use short access-token lifetimes. Stateless JWT access tokens generally cannot be individually erased without server-side revocation checks or very short expiry.

9. Failure and Debugging Decision Tree

Use the first failing boundary to locate the problem.



If login does not start

Check client /login, IDP_BASE_URL, redirect_uri, client_id, and whether port 5003 is reachable. The client should log the generated authorization URL.

If callback fails

Check the state value in the Flask session, the returned code, the exact redirect URI, and the /token response. A code can be used only once and expires after five minutes.

If API returns 401

Check Authorization: Bearer syntax, JWT three-part structure, kid, RS256, IDP /jwks reachability, issuer, audience resource_server, and exp. If exp is the problem, use refresh.

If API returns 403

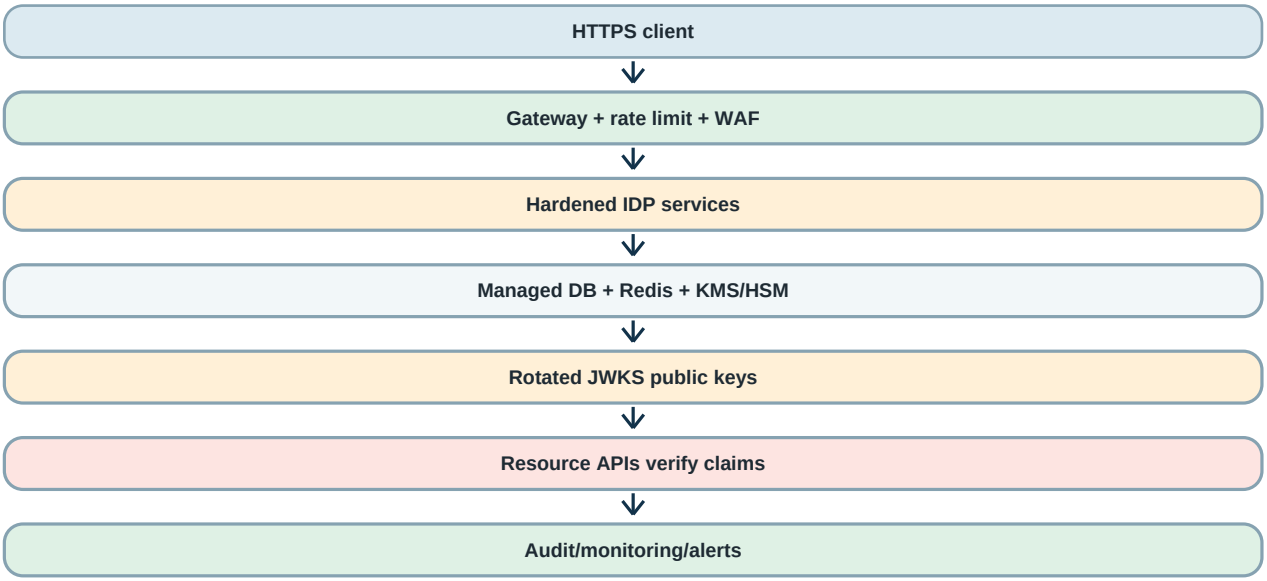
The token was accepted but the required scope was absent. Compare scope in the token with profile/read/write required by the decorator.

If data is empty

Authentication may be correct. Check the token subject and the username values in resource_database.db; the API intentionally filters documents to the authenticated subject.

10. Production Version of the Workflow

The same concepts remain, but the trust controls and operations become stronger.



Identity and credentials

Use a managed secrets system, encrypted database backups, password policy, breached-password checks, MFA/passkeys, password reset, account recovery, and audit events. Never commit private keys or client secrets.

Protocol hardening

Use HTTPS everywhere, exact registered redirect URIs, PKCE for public clients, secure HttpOnly SameSite cookies, strict CORS, CSRF protection where cookies are used, refresh-token rotation, revocation, key rotation, and discovery metadata that matches reality.

Authorization

Replace a small scope list with explicit policy, stable subject IDs, roles/permissions or policy-as-code, tenant boundaries, and endpoint tests for both allowed and denied actions.

Operations

Run multiple stateless instances, move from SQLite to a durable relational database, share short-lived state through Redis or a database, monitor latency/errors, alert on failed logins, and test recovery and key rollover.

Final explanation

A production-ready answer still sounds like this: the client sends the user to the IDP; the IDP authenticates and issues a code; the client exchanges it for audience-specific tokens; the API validates signature and claims; authorization policy checks the requested action; only then is the resource returned.